1. Project Title

SceneTrip: Movie-Based Trip Planner

2. Project Summary

Our project is a web application that generates a travel itinerary from a user's favorite films. Users will insert several movies they love into their favorite list. For each selected film, our system extracts its filming locations from a real movie dataset, normalizes them into city-level destinations, and then matches each city to real airports (IATA codes) using a global city–airport dataset. Next, our system will search a real flight route dataset to determine connectivity between the airports corresponding to the selected cities.

Based on this city-and-flight network, Our program automatically designs a trip that visits the film-related cities in an efficient order. We model the itinerary planning problem as a Traveling Salesman Problem (TSP)-style route optimization task: the "must-visit cities" are nodes, and the "flight time costs" are edges. The final output is a trip plan that includes an ordered list of cities to visit and recommended flight legs (source airport → destination airport) between consecutive stops.

3. Creative Component

Our creative component is the graph-based itinerary optimization that turns movie preferences into a feasible, optimized travel route. Instead of simply listing filming locations, our system builds a real-world connectivity graph by integrating three datasets: 1) film-to-location information, 2) city-to-airport mappings, and 3) flight route connections. This allows the application to generate an itinerary that is not only based on the user's movies, but also grounded in actual transportation links.

Technically, we will implement a TSP-style planning pipeline. The system first converts the user's favorite films into a set of target cities, then computes pairwise travel "costs" between these cities using flight-route information (e.g., direct connectivity, number of stops, or shortest reachable path in the route network). With these costs, we apply a practical TSP heuristic/approximation approach to produce a near-optimal visiting order. This is creative because it combines real datasets with non-trivial algorithmic planning to improve user experience: the user gets an itinerary that is automatically optimized and explainable, rather than a manual list of locations.

4. Usefulness

This website is useful because many people want to visit places they saw in a movie, but it is hard to find the information quickly. Our website makes it simple: search a movie → see real places → save them into a trip.

Some websites already list filming locations, but they usually do not let users build and manage a trip. Our app is different because users can save places and organize them into a personal trip plan.

5. Realness

We will use real datasets to build this application, and we will use at least three different data sources.

Data Source 1: Movie dataset

- From: Kaggle
- URL: https://www.kaggle.com/datasets/raedaddala/imdb-movies-from-1960-to-2023
- Format: CSV
- Data size: 63249 rows, 23 columns
- What it includes: id, title, duration, mpa, rating, votes, méta_score, description, movie_link, writers, directors, stars, budget, opening_weekend_gross, gross_worldwide, gross_us_canada, release_date, countries_origin, filming_locations, production_companies, awards_content, genres, languages

Data Source 2: City to airport dataset

- From: Kaggle
- URL: https://www.kaggle.com/datasets/samvelkoch/global-airports-iata-icao-timezone-geo

- Format: CSV

- Data size: 6,393 rows, 13 columns
- What it includes: AirportName, IATA, ICAO, TimeZone, City_Name, City_IATA, UTC_Offset_Hours, UTC_Offset_Seconds, Country_CodeA2, Country_CodeA3, Country_Name, GeoPointLat, GeoPointLong

Data Source 3: Flight dataset

- From: Kaggle

- URL: https://www.kaggle.com/datasets/divyanshrai/openflights-route-database-2014

- Format: CSV

- Data size: 67,663 rows, 9 columns

- What it includes: Airline, Airline ID, Source airport, Source airport ID, Destination airport, Destination airport ID, Codeshare, Stops, Equipment

6. Detailed Functionality

6.1 List of the functionality

- **Movie Input & Destination Extraction:** Users can search and insert several favorite films into their profile. Our system will retrieve each film's filming_locations from the movie dataset and standardize them into a set of destination cities.
- **City–Airport–Flight Matching:** For the destination cities, the system matches each city to real airports (IATA) using the city-to-airport dataset, and searches the flight route dataset to determine which airport pairs are connected.
- **TSP Trip Plan Generation & Management:** he system generates an optimized visiting order for the destination cities using a TSP-style method and outputs a trip plan with ordered cities plus recommended flight legs. Users can view, regenerate, or delete saved trip plans.

6.2 Low-fidelity UI mockup

6.3 Project work distribution

- **Member A (Frontend)**: Build the core pages and user flow, including Login/Profile, Movie Search & Favorites, Destination Preview , and Trip Plan Results pages; connect UI to backend APIs and support “Generate/Regenerate Trip” interactions.
- **Member B (Backend)**: Design and implement the database based on our schema (users, movies, cities/airports, flights, trip plans), load the three datasets, and build search APIs for favorites, city/airport lookup, flight connectivity queries, and trip plan storage.
- **Member C (Planning Algorithm)**: Implement the trip planning pipeline: convert favorite films into destination cities, compute travel cost/connectivity from the flight route network, run a TSP-style heuristic/approximation to produce an optimized visiting order, and return the itinerary to the backend.
- **Member D (Data Cleaning + Integration + Testing)**: Clean and standardize filming_locations → city mappings, handle edge cases, integrate the full pipeline end-to-end, and lead testing.